

KẾ HOẠCH ĐỒ ÁN TỐT NGHIỆP

Đề tài: Mạng xã hội tìm phòng trọ

1. TỔNG QUAN DỰ ÁN

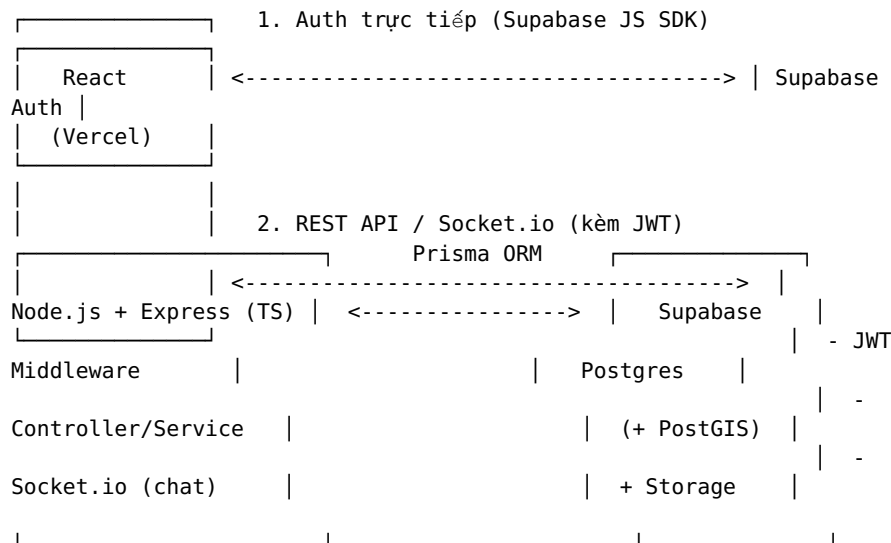
Mục tiêu: Xây dựng nền tảng web dạng mạng xã hội, kết nối người tìm trọ và chủ trọ, có tính năng định vị tự động, chat/video call realtime, chatbot hỗ trợ và diễn đàn chia sẻ.

Tech stack (đã đổi Backend sang JavaScript):

Thành phần	Công nghệ
Frontend	React (Vite hoặc Next.js)
Backend	Node.js + Express (TypeScript) — xử lý toàn bộ logic nghiệp vụ, expose REST API
ORM	Prisma (kết nối Postgres của Supabase, thay thế cho Spring Data JPA)
Database	Supabase (Postgres + PostGIS)
Auth	Supabase Auth (đăng ký/đăng nhập) — Backend Node.js chỉ xác thực lại JWT do Supabase phát hành
File/ảnh	Supabase Storage (gọi qua REST API hoặc @supabase/supabase-js từ backend)
Deployment Frontend	Vercel
Deployment Backend	Vercel Serverless Functions / Render / Railway — Node.js chạy được trực tiếp trên Vercel (khác với Java), nên có thể gộp chung 1 project Vercel nếu muốn
Realtime chat	Socket.io (thay cho WebSocket STOMP của Spring Boot)
Video call	WebRTC (signaling qua Socket.io) hoặc SDK có sẵn (Daily.co/Agora)
Bản đồ / định vị	Mapbox hoặc Goong Map / OpenStreetMap + Leaflet (frontend gọi trực tiếp)
Chatbot	Backend Node.js gọi OpenAI/Gemini API, trả kết quả về frontend
State management	Zustand hoặc React Query (TanStack Query)
UI framework	TailwindCSS + shadcn/ui
Validate dữ liệu	Zod hoặc Joi
Package manager	npm / pnpm

Vì sao đổi được dễ dàng: kiến trúc tổng thể (Frontend gọi API → Backend xác thực JWT → Backend truy vấn Postgres) không đổi, chỉ đổi ngôn ngữ implement tầng Backend từ Java sang

1.1. KIẾN TRÚC HỆ THỐNG (sau khi đổi sang JavaScript)



- **Bước 1:** Frontend đăng ký/đăng nhập trực tiếp với Supabase Auth (@supabase/supabase-js), nhận access_token (JWT).
- **Bước 2:** Mọi API nghiệp vụ (tạo phòng trọ, chat, tìm kiếm...) frontend gọi tới Node.js/Express kèm JWT ở header Authorization: Bearer <token>; Backend xác thực token rồi mới xử lý logic.
- **Database:** Backend dùng **Prisma** truy cập trực tiếp Postgres của Supabase cho toàn bộ business data; bảng auth.users do Supabase tự quản lý riêng, đồng bộ sang bảng users nghiệp vụ khi cần.

Cơ chế xác thực JWT phía Node.js (thay cho JwtAuthFilter của Spring Boot):

1. Lấy **JWT Secret** của project Supabase (Settings → API → JWT Secret).
2. Viết **Express middleware** dùng thư viện jsonwebtoken:
 - verify token bằng jwt.verify(token, SUPABASE_JWT_SECRET)
 - lấy claim "sub" (user id), "email"
 - gán vào req.user
 - next() nếu hợp lệ, trả 401 nếu sai/hết hạn
3. Áp middleware này cho tất cả route cần đăng nhập (router.use(authMiddleware)), route admin dùng thêm middleware kiểm tra role.
4. **Đồng bộ user:** khi user mới đăng ký qua Supabase, dùng **Supabase Database Webhook** gọi tới endpoint /internal/sync-user của Node.js để tạo bản ghi trong bảng users nghiệp vụ (Prisma) — hoặc middleware tự tạo record nếu chưa tồn tại (kiểm tra bằng sub).
5. **Role** (tenant / landlord / admin) lưu ở bảng users phía Postgres do Node.js/Prisma quản lý, không lưu trong Supabase Auth metadata — để backend toàn quyền kiểm soát phân quyền.

Cấu trúc project Node.js/Express đề xuất (layered architecture, tương đương Spring Boot cũ):

```
src/  
├── controllers/    (auth.controller.ts, room.controller.ts,
```

```
chat.controller.ts, admin.controller.ts...)
├─ services/      (room.service.ts, user.service.ts,
chat.service.ts, chatbot.service.ts...)
├─ repositories/  (hoặc dùng thẳng Prisma Client trong service
nếu project nhỏ)
├─ prisma/
├─   └─ schema.prisma (định nghĩa toàn bộ model, thay cho entity
Java)
├─ dto/           (Zod schema cho request/response, tương đương
DTO của Java)
├─ middlewares/   (auth.middleware.ts – verify JWT Supabase,
role.middleware.ts, error.middleware.ts)
├─ routes/        (auth.routes.ts, room.routes.ts,
chat.routes.ts...)
├─ sockets/       (chat.socket.ts – xử lý Socket.io cho
chat/video call signaling)
├─ config/        (env.ts, cors.ts, supabaseClient.ts)
└─ app.ts / server.ts
```

2. PHÂN TÍCH CHỨC NĂNG CHI TIẾT

2.1. Người dùng thường (người tìm trọ)

- **Định vị tự động:**
 - Lấy vị trí qua Geolocation API (trình duyệt) làm mặc định — ưu tiên navigator.geolocation, dùng IP-based geolocation (vd ipapi.co) làm phương án dự phòng khi user từ chối quyền định vị.
 - Reverse geocoding để suy ra Tỉnh/Thành → Quận/Huyện → Xã/Phường.
 - Backend (Node.js + Prisma raw query / \$queryRaw) dùng **PostGIS** để tính khoảng cách (ST_DWithin) và trả về danh sách bài đăng gần vị trí, sắp xếp theo khoảng cách.
 - Tự động cập nhật danh sách khi địa chỉ thay đổi (không cần bấm nút search) — debounce + gọi lại API khi tọa độ thay đổi.
- **Tìm kiếm nơi khác:** ô search có autocomplete địa danh (tích hợp Goong/Mapbox Places API).
- **Bộ lọc:** giá phòng, diện tích, số người ở, tiện ích (để xuất thêm để tăng tính thực tế).
- **Chat với người dùng khác** (cả chủ trọ và người dùng thường khác) qua Socket.io.
- Xem chi tiết bài đăng, lưu bài đăng yêu thích (để xuất thêm).

2.2. Chủ trọ

- **Tạo phòng trọ** với các trường: Tên nhà trọ, địa chỉ, số phòng, ID phòng, trạng thái (còn trống/đã thuê), nội dung mô tả phòng, mô tả nhà trọ, hình ảnh (Supabase Storage), giá, diện tích...
- **Chat room riêng cho từng nhà trọ:** tự động tạo khi tạo bài đăng, ID random (dùng uuid – package uuid của npm).
- **Share nhà trọ lên diễn đàn:** đăng bài chia sẻ kèm link tới phòng trọ.
- Quản lý danh sách phòng đã đăng, chỉnh sửa trạng thái từng phòng (trống/đã có người).

2.3. Admin

- Quản lý người dùng (khóa/mở tài khoản, phân quyền).
- Quản lý bài đăng (duyet/gỡ bài vi phạm).
- Quản lý báo cáo (report) từ người dùng.
- Thống kê tổng quan (số bài đăng, số user, hoạt động chat...).

2.4. Chức năng chung của Website

- Đăng nhập/Đăng ký (qua Supabase Auth): bắt buộc **email + số điện thoại**, mật khẩu; các trường như CCCD sẽ yêu cầu bổ sung ở lần đăng nhập đầu tiên (modal “hoàn thiện hồ sơ”, lưu vào bảng nghiệp vụ qua API Node.js).
 - Tự chọn role khi đăng ký — cần làm rõ: cho phép 1 tài khoản có cả 2 role không, hay chọn 1 lần duy nhất? (xem mục 5).
 - Chat 1-1, chat nhóm riêng tư, video call — qua Socket.io.
 - Chatbot hỗ trợ (trả lời FAQ, hướng dẫn dùng web, gợi ý tìm phòng) — Node.js gọi LLM API.
 - Diễn đàn chia sẻ bài đăng.
-

3. THIẾT KẾ CƠ SỞ DỮ LIỆU (Supabase/Postgres, quản lý qua Prisma)

// schema.prisma (rút gọn, minh hoạ)

```
model User {
  id          String    @id // = auth.users.id bên Supabase
  email       String    @unique
  phone       String    @unique
  fullName    String?
  cccd        String?
  avatarUrl   String?
  role        Role      @default(TENANT)
  createdAt   DateTime  @default(now())

  boardingHouses BoardingHouse[]
  posts          Post[]
  messages       Message[]
}

enum Role {
  TENANT
  LANDLORD
  ADMIN
}

model BoardingHouse {
  id          String    @id @default(uuid())
  ownerId     String
  owner       User      @relation(fields: [ownerId], references:
[id])
  name        String
  address     String
  lat         Float
  lng         Float     // dùng cùng cột geography(Point) qua PostGIS
  tầng DB
  description String?
  chatRoomId  String?   @unique
  rooms       Room[]
  createdAt   DateTime  @default(now())
}

model Room {
  id          String    @id @default(uuid())
  boardingHouseId String
  boardingHouse BoardingHouse @relation(fields: [boardingHouseId],
references: [id])
  roomCode    String
  status      RoomStatus @default(AVAILABLE)
  price       Float
  area        Float
  content     String?
  images      Json?
}
```

```

enum RoomStatus {
  AVAILABLE
  OCCUPIED
}

model Post {
  id      String   @id @default(uuid())
  authorId String
  author   User     @relation(fields: [authorId], references: [id])
  roomId   String?
  content  String
  images   Json?
  createdAt DateTime @default(now())
}

model ChatRoom {
  id      String   @id @default(uuid())
  type     ChatType
  createdAt DateTime @default(now())
  members  ChatRoomMember[]
  messages Message[]
}

enum ChatType {
  DIRECT
  GROUP
  BOARDING_HOUSE
}

model ChatRoomMember {
  chatRoomId String
  userId      String
  chatRoom    ChatRoom @relation(fields: [chatRoomId], references:
[id])

  @@id([chatRoomId, userId])
}

model Message {
  id      String   @id @default(uuid())
  chatRoomId String
  chatRoom ChatRoom @relation(fields: [chatRoomId], references:
[id])
  senderId String
  sender   User     @relation(fields: [senderId], references:
[id])
  content  String
  type     String   @default("text") // text | image | call-log
  createdAt DateTime @default(now())
}

model Report {
  id      String   @id @default(uuid())
  reporterId String
  targetType String
  targetId String
  reason   String
  status    String @default("pending")
  createdAt DateTime @default(now())
}

```

PostGIS: Prisma chưa hỗ trợ kiểu geography sẵn trong schema, nên cần: (1) tạo cột tọa độ bằng raw SQL migration (ALTER TABLE ... ADD COLUMN geog geography(Point,4326)), (2) dùng prisma.\$queryRaw để chạy query ST_DWithin khi tìm phòng trọ gần vị trí.

4. DANH SÁCH TRANG / MÀN HÌNH CHÍNH

1. Trang chủ (feed bài đăng theo vị trí)
2. Trang tìm kiếm/bản đồ
3. Trang chi tiết phòng trọ
4. Trang đăng nhập / đăng ký (chọn role)
5. Trang hoàn thiện hồ sơ (lần đầu đăng nhập)
6. Dashboard chủ trọ (quản lý nhà trọ/phòng)
7. Form tạo/sửa phòng trọ
8. Trang chat (danh sách hội thoại + khung chat + video call)
9. Trang diễn đàn
10. Trang chatbot hỗ trợ (widget nổi hoặc trang riêng)
11. Trang admin (quản lý user, bài đăng, report, thống kê)
12. Trang cá nhân / hồ sơ người dùng

5. NHỮNG ĐIỂM CẦN LÀM RÕ TRƯỚC KHI CODE

- Một tài khoản có được vừa là “người tìm trọ” vừa là “chủ trọ” không, hay phải chọn cố định 1 role?
- Video call dùng giải pháp nào: tự build WebRTC qua Socket.io signaling (khó, tốn thời gian, phù hợp để “ăn điểm” kỹ thuật) hay dùng SDK có sẵn (nhANH, ổn định hơn, phù hợp nếu deadline gấp)?
- Chatbot: rule-based đơn giản hay tích hợp LLM (cần tính chi phí API)?
- Có cần duyệt bài đăng trước khi hiển thị công khai không (kiểm duyệt nội dung)?
- Phạm vi “gần vị trí” mặc định là bao nhiêu km, có cho user tùy chỉnh không?

(Nên đưa các câu này vào phần “Phạm vi đề tài” của báo cáo để giới hạn khối lượng công việc hợp lý với thời gian làm đồ án.)

6. LỘ TRÌNH PHÁT TRIỂN ĐỀ XUẤT (theo tuần, ~12 tuần)

Giai đoạn	Thời gian	Nội dung
1. Chuẩn bị	Tuần 1	Đề cương, thiết kế DB (ERD), thiết kế UI/UX (Figma), khởi tạo repo Frontend + repo Backend Node.js, setup Supabase (Auth + Postgres + Storage), setup Vercel
2. Backend nền tảng	Tuần 2	Setup Express + TypeScript, cấu hình Prisma kết nối Supabase Postgres, định nghĩa schema.prisma, cấu trúc controller/service/route, cấu hình CORS
3. Auth & User	Tuần 3	Frontend tích hợp Supabase Auth SDK (đăng ký/đăng nhập/chọn role); Backend viết middleware verify JWT

		Supabase (jsonwebtoken), đồng bộ user mới sang bảng nghiệp vụ, API hoàn thiện hồ sơ
4. Chức năng chủ trọ	Tuần 4-5	API tạo/sửa/xoá phòng trọ (Express + Prisma), upload ảnh lên Supabase Storage qua backend, quản lý trạng thái phòng
5. Định vị & tìm kiếm	Tuần 6	Geolocation ở frontend, API tính khoảng cách bằng PostGIS raw query (\$queryRaw + ST_DWithin), API search nơi khác
6. Chat realtime	Tuần 7-8	Tích hợp Socket.io cho chat 1-1, chat room theo nhà trọ, nhóm chat riêng tư
7. Video call	Tuần 9	WebRTC signaling qua Socket.io, hoặc tích hợp SDK có sẵn
8. Diễn đàn + Chatbot	Tuần 10	API diễn đàn (Express), tích hợp chatbot (Node.js gọi LLM API)
9. Trang Admin	Tuần 11	API + giao diện quản lý user, bài đăng, report, thống kê
10. Kiểm thử & tối ưu	Tuần 12	Unit test (Jest/Vitest) cho service, test tích hợp (Supertest), sửa bug, tối ưu hiệu năng, responsive
11. Hoàn thiện báo cáo	Tuần 13	Viết báo cáo (nhấn mạnh kiến trúc backend Node.js/Express), chuẩn bị slide, demo

7. ĐỀ XUẤT MỞ RỘNG (nếu còn thời gian, giúp đồ án nổi bật hơn)

- Đánh giá/review nhà trọ (rating).
- Thông báo (notification) real-time khi có tin nhắn mới, phòng vừa đăng gần khu vực quan tâm (dùng Socket.io kết hợp).
- Gợi ý phòng trọ theo AI (dựa trên lịch sử tìm kiếm).
- Xác thực bài đăng thật (yêu cầu ảnh CCCD/giấy tờ nhà đất khi đăng ký làm chủ trọ) để tăng độ tin cậy.

8. TÓM TẮT NHỮNG THAY ĐỔI SO VỚI PHIÊN BẢN JAVA

Hạng mục	Trước (Java)	Sau (JavaScript)
Ngôn ngữ Backend	Java	Node.js + TypeScript
Framework Backend	Spring Boot	Express
ORM	Spring Data JPA / Hibernate	Prisma

Xác thực JWT	Spring Security + JWT Filter	Middleware Express + thư viện jsonwebtoken
Realtime	Spring WebSocket (STOMP)	Socket.io
Build tool	Maven/Gradle	npm/pnpm
Testing	JUnit	Jest / Vitest + Supertest
Deploy Backend	Cần VPS/Render riêng (Vercel không chạy Java)	Có thể deploy trực tiếp trên Vercel (Serverless Functions) hoặc Render/Railway
Kiến trúc tổng thể	Không đổi	Không đổi
Auth (Supabase Auth + verify token ở backend)	Giữ nguyên	Giữ nguyên
Database & schema	Giữ nguyên	Giữ nguyên (chuyển từ JPA entity sang Prisma schema)

Lợi ích chính khi chuyển sang JavaScript: đồng bộ ngôn ngữ Frontend-Backend (đều là JS/TS), dễ deploy chung trên Vercel, giảm thời gian học thêm ngôn ngữ mới nếu nhóm đã quen React. **Đánh đổi:** hệ sinh thái Java (Spring) có sẵn nhiều tính năng enterprise (validation, security annotation...) mà Node.js cần cấu hình thủ công hơn — nhưng với quy mô đồ án tốt nghiệp, Node.js/Express hoàn toàn đáp ứng tốt.